

Linq and extension methods

Linq

- **L**anguage **I**ntegrated **Q**uery
- Process Collections
 - all classes inheriting from IEnumerable (List, Array, Dictionary, ...)
- SQL-like usage (filtering, sorting, summing, ...)
- lambda expressions (=>)
- concatenate single Linq queries
- do not change the original collection
 - Assignment to the original collection to "Apply" changes

Structure

```
List<int> list = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9 };  
    list.Where(x => x % 2 == 0).ToList();
```

Originale
Collection

Linq-
Funktion

Lambda-Expression

Linq-
Funktion

- Find all even numbers and create a new list
- set a condition with variable name (x) and ==>
- finally use ToList() to convert the IEnumerable<int> resulting from Where(...) to a List<int>.

System.Linq.Enumerable.ToList<TSource> returned		Count = 4	System.Collections.Generic.List<int>
	[0]	2	int
	[1]	4	int
	[2]	6	int
	[3]	8	int

Basic Linq functions

```
List<int> list = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

Name	Description	Result
Average	Returns the average of the list	5
Min/Max	Returns the smallest/largest value of the list	1, 9
Sum	Returns the sum of the list	45
First	Returns the first element of the list, throws an exception if no element is found	1
FirstOrDefault	First element, null if no element is found	1
Last	Returns the last element of the list, throws an exception if no element is found	9
LastOrDefault	Last element, null if no element is found	9

Linq notation comparison

```
List<int> ints = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

```
List<int> even = new List<int>();  
foreach (int i in ints)  
    if (i % 2 == 0)  
        even.Add(i);
```

```
List<int> evenSQL = (from i in ints where i % 2 == 0 select i).ToList();
```

```
List<int> evenLinq = ints.Where(e => e % 2 == 0).ToList();
```

Linq with object list

```
List<Car> cars = new List<Car>();
```

Name	Description	Result
Where	Filters the list according to one or more criteria	IEnumerable<Car>
Select	Projects the list to a new list according to a criteria	IEnumerable<?>
Contains	Checks if the list contains an element	bool
OrderBy and OrderByDescending	Returns a new list sorted (ascending or descending) by a criteria	IEnumerable<Car>
All/Any	Checks if all/at least one element in the list meets the specified condition	bool
Count/LongCount	Count the elements in the list based on one/more criteria	int/long
MinBy/MaxBy	Returns the object with the smallest/largest value based on a criteria	Car
Chunk	Splits the list to parts with the specified size	IEnumerable<Car[]>

Linq with object list (2)

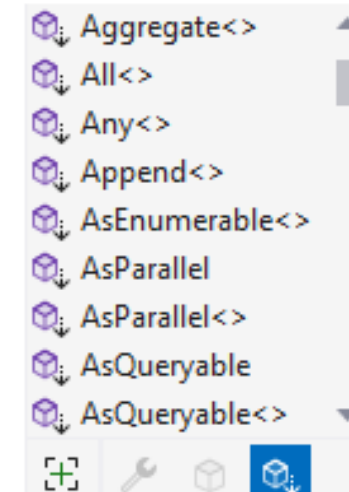
Name	Description	Result
Skip/Take	Skips X elements, takes the next X elements. Use in combination to take parts of the list	IEnumerable<Car>
Reverse	Creates a new list that is the original list only inverted	IEnumerable<Car>
Concat	Combines two lists into a new list	IEnumerable<Car>
Prepend/Append	Creates a new list and appends the element to the beginning/end	IEnumerable<Car>
Except	Returns a new list with elements that are only in one of the two lists	IEnumerable<Car>
Intersect	Returns the intersection between the two lists	IEnumerable<Car>
Zip	Connects two lists to one list of tuples	IEnumerable<(? Name, Car c)>
GroupBy	Groups the list according to a criteria	IEnumerable<IGrouping<?, Car>>
Aggregate	Applies a function to an aggregator for each element	? (Type of the aggregator)

Conversion functions

- Convert original collection into another form
 - List $\leftrightarrow$ Array
 - IEnumerable List
 - IEnumerable<IGrouping<?, Car>> (GroupBy) $\rightarrow$ Dictionary
- ToArray
- ToDictionary
- ToHashSet
- ToList
- AsEnumerable

Extension methods

- all Linq methods are extension methods
- attach any methods to types
- marked with cube + arrow down or with (extension) at method preview
- Atructure
 - must be defined in a static class
 - must themselves also be static
 - with this <type> refer to a type



Show only Extension methods (Alt+X)

(extension) IEnumerable<int>
Filters a sequence of values ba

Custom extension method

```
public static int GetCrossSum(this int num)
{
    int sum = 0;
    string numAsString = num.ToString();
    for(int i=0;i<numAsString.Length; i++)
    {
        sum += (int) char.GetNumericValue(numAsString[i]);
    }
    return sum;
}
```

This indicates for which class an extension method is defined

```
Console.WriteLine(10232.GetCrossSum()); //Gives 8
```